National University of Computer and Emerging Sciences

Lab Manual

Data Analysis and Visualization

Lab 06

Instructor	Rida Mahmood
Class	BDS-5A/5D
Semester	Fall 2023

Fast School of Computing

FAST-NU, Lahore, Paki

References:

Lecture 11 and Lecture 12 of Theory

Resources: (Stemming and Lemmatization)

https://www.turing.com/kb/stemming-vs-lemmatization-in-python

Reading Material:

<u>Python list, tuple and dictionary example</u>
Create a file called list merge.py. Type the following code and fill in the missing parts (< ... >). Create a dictionary result, where the keys are the values from some list, and values from some tuple. Use list comprehension or standard loop.

some_list = ["first_name", "last_name", "age", "occupation"] some_tuple = ("John",

"Holloway", 35, "carpenter")

result = <your code>

print(result)

{'first_name': 'John', 'last_name': 'Holloway', 'age': 35, 'occupation': 'carpenter'}
Submission: Add the code where indicated and submit merge.py to Moodle.

<u>Lexical Analysis: tokenization</u>

Word tokenization: A sentence or data can be split into words using the method word tokenize(): from

nltk.tokenize import sent_tokenize, word_tokenize

data = "All work and no play makes jack a dull boy, all work and no play" print(word_tokenize(data))

This will output:

['All', 'work', 'and', 'no', 'play', 'makes', 'jack', 'dull', 'boy', ',', 'all', 'work', 'and', 'no', 'play']

All of them are words except the comma. Special characters are treated as separate tokens.
Sentence tokenization:
The same principle can be applied to sentences. Simply change the to
sent_tokenize() We have added two sentences to the variable data:

from nltk.tokenize import sent_tokenize, word_tokenize

data = "All work and no play makes jack dull boy. All work and no play makes jack a dull boy."

print(sent_tokenize(data))

Outputs:

['All work and no play makes jack dull boy.', 'All work and no play makes jack a dull boy.']

Storing words and sentences in lists

If you wish to you can store the words and sentences in lists:

from nltk.tokenize import sent_tokenize, word_tokenize

data = "All work and no play makes jack dull boy. All work and no play makes jack a dull boy."

phrases = sent_tokenize(data) words =

word_tokenize(data) print(phrases)

print(words)

Stop word removal

English text may contain stop words like ‘the’, ‘is’, ‘are’. Stop words can be filtered from the text to be processed. There is no universal list of stop words in NLP research, however the NLTK module contains a list of stop words. Now you will learn how to remove stop words using the NLTK. We start with the code from the previous section with tokenized words.

```
from nltk.tokenize import sent_tokenize, word_tokenize from

nltk.corpus import stopwords # We imported auxiliary corpus

# provided with NLTK data = "All work and no play makes

jack dull boy. All work and no play makes jack a dull boy."

stopWords = set(stopwords.words('english')) # a set of

English stopwords words = word_tokenize(data.lower())

wordsFiltered = []

for w in words:

if w not in stopWords:

wordsFiltered.append(w)
```

Stemming

A word stem is part of a word. It is sort of a normalization idea, but linguistic. For example, the stem of the word waiting is wait. Given words, NLTK can find the stems. Start by defining some words:

words = ["game","gaming","gamed","games"] We import the module:

from nltk.stem import PorterStemmer

from nltk.tokenize import sent_tokenize, word_tokenize

And stem the words in the list using:

from nltk.stem import PorterStemmer from nltk.tokenize import sent_tokenize,

word_tokenize

words = ["game","gaming","gamed","games"]

ps = PorterStemmer() for word in words:

print(ps.stem(word)) You can do word stemming for

sentences too:

from nltk.stem import PorterStemmer from nltk.tokenize

import sent_tokenize, word_tokenize

ps = PorterStemmer() sentence = "gaming, the gamers play games"

words = word_tokenize(sentence)

for word in words:

print(word + ":" + ps.stem(word))